SE1 Summative 3 Reviewer: Modules 5, 6 and 7

CS0025 Software Engineering 1. Study notes for Module 5 (OOAD, Procedural Programming, Methodologies), Module 6 (Development Strategies, UML), and Module 7 (Strategic Trends). The blue boxes are memory aids I made; they are not from the slides.

What Is Inside

- **Part 1, Module 5:** Object-Oriented Analysis and Design, Procedural Programming, then Methodologies and the six system development models.
- **Part 2, Module 6:** Software Development Strategies, then Unified Modeling Language (UML).
- **Part 3, Module 7:** AI-Native Engineering, LLM-Based Applications and Agents, GenAI Platform Engineering, Talent Density, Open GenAI Ecosystems, and Green Software Engineering.
- Slide mistakes are fixed in the **Watch Out** boxes: the correct fact first, then what the slide says.
- Read the cram sheet last, right before the exam.

Fast Memory Map

Topic	Hook
Object	Pretty Big Spoons = Properties, Behaviors, States
OOA five activities	Fast Owls Do Eat Insects
OOAD good and bad	Easy Eggs, Ill Tummies
Parameter passing	Very Rich Rabbits Visit Nashville
MSF phases	Every Panda Dances, Sleeps, Dreams
Nine criteria	4 + 1 + 3 + 1: Pretty Pandas Cook Cakes, Responsiveness, Team Management Training, Probability
Six models	Wise Ice Dragons Vanish Silently, Perhaps
Spiral quadrants	Our Rabbits Dance Politely
Structural UML	Cats Obey Calm Dogs
Behavioral UML	A Unicorn Tripped, Sliding Sideways, Crying
AI-native key ideas	Dogs Like Apple Eating
LLM agent applications	Some Tigers Read Wild Poems
Talent density	Hire, Develop, Culture, Manage
Green practices	Always Open Pretty Skies

Part 1: Module 5

Module 5, Chapter 1: Object-Oriented Analysis And Design

Objects

In computer science an **object** can be almost anything in a programmatic sense: a variable, a data model, a function, a class, or a method. In **object-oriented programming (OOP)** an object is an **instance** that usually represents a real-world thing. It has **properties** (characteristics), **behaviors** (methods), and **states** (data).

Memory Aid
An object = **P-B-S**: **P**roperties, **B**ehaviors, **S**tates. Say it as "Pretty Big Spoons".

Watch Out
The slide says an object has properties, behaviors, and states. Wrong choices on the quiz were "functions and loops only", "inputs and outputs only", and "hardware and network devices".

Object-Oriented Analysis (OOA)

OOA is an **iterative stage of analysis** during the software development life cycle. It **models the functional requirements** of the software while staying **completely independent of any implementation requirements**. It looks at everything through the lens of objects and combines behaviors, characteristics, and states into **one analysis process** instead of splitting them into separate stages.

The Five Activities Of OOA

1. **Find and define** the objects.
2. **Organize** the objects.
3. **Describe how the objects interact** with one another.
4. Define the **external behavior** of the objects.
5. Define the **internal behavior** of the objects.

Memory Aid
Fast Owls Do Eat Insects = **F**ind, **O**rganize, **D**escribe interactions, **E**xternal behavior, **I**nternal behavior.
Chunk it as **2 + 1 + 2**: objects (1 and 2), interaction (3), behavior outside then inside (4 and 5).

Object-Oriented Design (OOD)

OOD is an **extension of OOA**. The difference is that it **considers implementation constraints**. An analyzed object or object model has to be designed and implemented for real, which usually needs constraints such as **software or hardware platforms, time and budget limits, performance requirements, and developer aptitude**.

	OOA	OOD
Question it answers	What must the system do?	How will it be built under real limits?
Focus	Functional requirements, modeled with objects	The analyzed objects plus implementation constraints
Implementation	Independent of it	Considers it (platform, time, budget, performance, skills)
Relationship	Comes first	Extends the OOA result

Memory Aid
A = Analyze what, D = Design how. If a quiz option says OOD "eliminates the identified objects" or "avoids software and hardware requirements", it is wrong: OOD keeps the objects and adds the constraints.

Object-Oriented Analysis And Design (OOAD)

OOAD is a **technical method of analyzing and designing an application based on the system's object models** (the logical components of the system that interact with one another). Development is normally split into loose, abstract stages. The limits of those stages, together with the strict step-by-step staging of the waterfall model, led to the rise of **iterative models like OOAD**.

Watch Out
OOAD is a method based on object models. It is **not** a programming language for databases, not only a testing technique, and not a model for managing computer networks.

Advantages And Disadvantages Of OOAD

Point	Kind	What The Slide Says
Encourages encapsulation	Advantage	Everything revolves around objects, so systems are truly independent and self-contained. Objects can be mixed and matched as needed.
Easy to understand	Advantage	Based on real-world objects, so everyone on the team (and customers or non-technical users) quickly understands what an object name or behavior means.
Ill-suited to procedural applications	Disadvantage	Hard to apply OOAD in a procedural language or to non-object business logic. Procedural apps focus on scope and modularity; OO apps focus on objects that simulate the real world.
Too complex for simple applications	Disadvantage	The simpler the needs, the more likely a less-structured procedural approach is the better fit.

Memory Aid
2 good + 2 bad: **Encapsulation and Easy** are the good ones (**E, E**). **Ill-suited and Too**

complex are the bad ones (**I, T**). "Easy Eggs, Ill Tummies."

Module 5, Chapter 2: Procedural Programming

What Procedural Programming Is

Procedural programming uses a **list of instructions** that tell the computer what to do **step by step**. It relies on **procedures** (routines or subroutines), and each procedure holds a series of computational steps. It is also called **imperative programming**, and procedural languages are known as **top-down languages**. It is intuitive because it looks like how you would expect a program to work.

Before OOP became popular, the most common approach was **procedural, or structured, programming**. Structured programming uses **top-down design**: it breaks a problem into a series of more and more specific functions. It needs **well-defined flow control**. Development starts by **identifying the problem domain** and then defining a top-level function, which is split into more specific procedures.

Key Terms

Term	Meaning
Predefined function	An instruction identified by a name, usually built into the higher-level language and taken from a library or registry rather than from the program itself.
Local variable	Declared inside a method and limited to that local scope. Using it outside the method breaks the code.
Global variable	Declared outside every function, so **all functions** can use it.
Modularity	Grouping separate tasks and finishing them one after another until the larger task is complete.
Parameter passing	The mechanism for passing parameters to functions, subroutines, or procedures. Five ways: pass by **value**, **reference**, **result**, **value-result**, and **name**.

Memory Aid
Local = **L**imited to its method. Global = **G**oes everywhere.
Parameter passing, in order: **Value, Reference, Result, Value-result, Name** ("Very Rich Rabbits Visit Nashville").

Advantages And Disadvantages

Advantages	Disadvantages
Excellent for **general-purpose** programming	Code is **harder to write**
Simple code and **easy to implement** in compilers and interpreters	Code is often **not reusable**, so it may need recreating for another application
Plenty of books and courses on tested algorithms	**Hard to relate to real-world objects**, unlike OOP
Source code is **portable** (can target a different CPU)	Importance goes to the **operation rather than the data**, which is a problem in data-

	sensitive cases
Code can be **reused** in different parts of the program without copying	Data is **exposed to the whole program**, so it is less security friendly
Memory requirement is reduced	
Program flow is **easy to track**	

Procedural Versus Object-Oriented Programming

Point	Procedural	Object-Oriented
Program is divided into	Functions	Objects
Approach	Top-down	Bottom-up
Access specifiers	None	Private, public, protected
Adding data and functions	Not easy	Easy
Data hiding	No proper way, so less secure	Yes, so more secure
Overloading	Not possible	Possible
What matters more	Function over data	Data over function
Based on	The unreal world	The real world
Flow control	Function calls	Conditional statements and loops
Iterating collections	Recursion	Loops
Side effects	None of its functions	Methods can have side effects
Focus	"What you are doing"	"How you are doing it"

Memory Aid
Hook: **Procedural = Procedures, top-Down, Data exposed. OOP = Objects, bottom-Up, data hidden.** Procedural: **F**unctions, **T**op-down, **F**unction first. OOP: **O**bjects, **B**ottom-up, **D**ata first.

Watch Out
Correct: procedural programming is **imperative** (the slide's own definition page says so). (The comparison table on the slide lists procedural as "declarative" and OOP as "imperative", which contradicts that. If a question is copied straight from the table, answer what the table says, but remember the correct idea is procedural = imperative.)

Module 5, Chapter 3: Methodologies And The System Development Process

A **system development methodology** is the set of **steps used to form, plan, and control the process of developing an information system**. It gives guidelines for every activity in system development and includes specific **models, tools, and techniques**. The word **process** is a synonym for methodology, and the process is supported by **documentation**. The chapter cover diagram is the **Microsoft Solutions Framework (MSF)**, a circular model with five phases, each ending in a milestone.

MSF Phase	Exit Milestone
Envisioning	Vision/Scope Approved
Planning	Project Plan Approved
Development	Scope Complete
Stabilizing	Release Readiness
Deployment	Deployment Complete

Memory Aid
MSF phases: **E-P-D-S-D** = **E**nvision, **P**lan, **D**evelop, **S**tabilize, **D**eploy ("Every Panda Dances, Sleeps, Dreams"). Milestones in the same order: Vision, Plan, Scope, Release, Deployment.

The Nine Criteria For Comparing Methodologies

#	Criterion	The Question It Asks
1	**Processes definition**	Does the methodology assume every step is well known with no room for change, or is it flexible in all stages?
2	**Final product determination**	Is the final product defined early in planning, or during the project and close to closure?
3	**Project cost**	At what stage is the cost estimated?
4	**Project completion date**	Is the schedule of deliverables estimated up front or as the project progresses?
5	**Responsiveness to environment**	How much flexibility is allowed for changes in environment, technology, competition, and others?
6	**Team dynamic and creativity**	Do predefined linear steps block creativity? Does it allow teamwork and interaction?
7	**Role of upper management**	Is management an obstacle, or does it empower the team by removing obstacles?
8	**Training and knowledge**	Does the methodology allow training and knowledge transfer, or limit what a member can do and learn?
9	**Probability of success**	How likely is success given a certain degree of complexity and unpredictability?

Memory Aid
Chunk **4 + 1 + 3 + 1**: the project basics (**P**rocess, **P**roduct, **C**ost, **C**ompletion date), then

Responsiveness, then the people three (**T**eam, **M**anagement, **T**raining), then **P**robability of success.
First four: "**P**retty **P**andas **C**ook **C**akes."

Tools And Techniques

	What It Is	Examples
Tools	Include the actual **programming language** the application is created in, plus supporting tools.	Project management software, **IDE**, code generators, **CASE**
Techniques	Also called **best practices**: guidelines or policies that help developers and analysts complete a task or activity in an SDLC phase.	Use-case modeling, software testing, user-interviewing techniques, relational database design techniques

Memory Aid
Tools = things you **use** (language, IDE). Techniques = ways you **work** (best practices). Four supporting tools: **P-I-C-C** = **P**roject management, **I**DE, **C**ode generators, **C**ASE.

Watch Out
Correct: **CASE = Computer-Aided Software Engineering**. (The slide writes "Computer-Aided System Engineering", so if a question quotes the slide, the slide's wording is the one in the answer key.)

The Six System Development Models

The slide lists **six** models: **Waterfall, Iterative, DevOps, V-Model, Spiral, and Prototyping**.

Memory Aid
W-I-D-V-S-P: "**W**ise **I**ce **D**ragons **V**anish **S**ilently, **P**erhaps."

Model	Core Idea	Best For / Trade-Off	Diagram Detail
Waterfall	One of the **oldest** SDLC models. **Rigid** structure: **all requirements defined at the very start**, then design and development. Testing against the original requirements comes after development, then rework.	Out of favor in recent years, but strong for certain projects, **especially government contractors**.	Stages cascade down: Requirements (product requirements document), Design (software architecture), Implementation (software), Verification, Maintenance.
Iterative	Takes waterfall and **cycles it several times in small increments**. Each	Shares many goals with **agile**, but **external customers**	Loop through planning, requirements, design, implementation, verification,

	step becomes several **mini-projects** that add value as the product evolves.	**are less involved** and the **scope of each increment is normally fixed**.	evaluation. Entry: initialization and planning. Exit: implementation then deployment.
DevOps	One of the **newest** methodologies. Combines software **Dev**elopment and IT **Op**erations to **shorten the SDLC** and give **continuous delivery with high software quality**.	Brings development and operations teams together to streamline delivery and support. Changes become more fluid and organizational risk drops. Needs **flexible resources**.	Venn diagram: DevOps sits where **Development, QA, and Operations** overlap.
V-Model	An **evolution of the classic waterfall**. The steps are **flipped upwards after the coding phase**. Very strict: the next phase starts only when the previous is complete.	Inflexible and higher risk, **not recommended for small projects**, but **easier to manage and control**. Good where requirements are **static and clearly stated** and **early testing** is wanted.	Left side down: Concept of Operations, Requirements and Architecture, Detailed Design. Bottom: Implementation. Right side up: Integration/Test/Verification, System Verification and Validation, Operation and Maintenance.
Spiral	Lets teams adopt **multiple SDLC models based on the risk patterns** of the project. A **blend of iterative and waterfall**.	The challenge is **knowing when to move on to the next phase**. Good for businesses unsure of requirements or expecting major edits in **mid to high-risk** projects.	Four quadrants: 1 Determine objectives, 2 Identify and resolve risks, 3 Development and test, 4 Plan the next iteration. Radius = cumulative cost.
Prototyping	Build an **early model** of the new system. It has **limited functionality and is not thoroughly tested**, but gives customers a sense of what is coming.	**Feedback** is gathered and used through the rest of the SDLC. Works well for **emerging industries or new technologies**.	Requirements, quick design, build prototype, user evaluation (loops with refining the prototype), implement and maintain.

Memory Aid
One-line hooks: Waterfall = **rigid, requirements first**. Iterative = **small repeated increments**. DevOps = **Dev + Ops, continuous delivery**. V-Model = **waterfall bent up, test every stage**. Spiral = **risk-driven**. Prototyping = **early model, feedback**.
Spiral quadrants: **O-R-D-P** = **O**bjectives, **R**isks, **D**evelop and test, **P**lan next ("Our Rabbits Dance Politely").

Watch Out
Iterative vs Agile: both deliver in increments, but iterative has **fixed increment scope** and **less customer involvement**. V-Model vs Waterfall: V-Model **pairs every build stage with a test stage**. Waterfall tests only after implementation.

Watch Out
Correct: "flexibility **than** it offers" and "**Businesses** that are not sure". (The slides say "flexibility that what it offers" and "Business that aren't sure".) The prototyping diagram feeds refinement back into the prototype build, which is what the loop means.

Part 2: Module 6

Module 6, Chapter 1: Software Development Strategies

This chapter is about **choosing** a development strategy, not about one perfect method. The slides list the reasons to choose carefully, and warn against the most common mistake.

Why Strategy Matters

- **Keep up** with the rapidly evolving industry.
- **Develop software realistically.**
- Consider that **every project and team is unique.**
- **Find the right solution.**
- Identify **useful techniques** relevant to the situation.
- **Choose the appropriate development model.**

Memory Aid
Six verbs: **Keep up, Be realistic, Remember uniqueness, Find, Identify, Choose.** The last three are the decision path: find the solution, pick techniques, choose the model.

The Big Ideas

Idea	What The Slides Say
Misconception	Adopting some methodology does **not** guarantee a project's success over another methodology or over no single strategy at all.
Why people believe it	Assuming what worked in one case will **automatically** work in another. Common in the software industry and with misunderstood concepts like agile.
Reality of teams	Most teams are **not industry experts**. Most projects do not use groundbreaking tools. They have budgets and timelines and often must interface with **less-than-ideal legacy systems**.
Copying a method	Adopting a way of doing things because it looks good, **without understanding how and why**, often ends in disaster.
Why projects fail hard	Applying a **one-size-fits-all** approach without enough analysis of the specific goals, problems, challenges, and exceptions.
What proves it will work	Only a properly evaluated situation, with solutions **thoroughly tested and sanity-checked**. Otherwise there is no real evidence things will go well.
Many processes to choose from	Pick tools and techniques from each process, as long as you learn what they mean and whether they are **compatible** and make sense.
No single perfect solution	There is no "one" perfect solution, but the **best possible one for the given problem** can be found.
Inventing something new	Often works: base it on a thorough understanding of the problem and the shortcomings of known methods, using a toolbox of abstractions and generalizations to **re-assemble** a strategy from

	proven components.
Small closely knit team	High **trust and autonomy** across the board.
Flexibility	A model or methodology must have **flexibility built in**, or things pile up the moment anything goes wrong.

Memory Aid
One sentence for the chapter: **"No one-size-fits-all: understand the problem, test the plan, keep it flexible."** If an option says one methodology guarantees success, it is the trap.

Module 6, Chapter 2: Unified Modeling Language (UML)

UML is a **general-purpose, developmental modeling language** in software engineering, intended to give a **standard way to visualize the design of a system**. It prescribes a standard set of **diagrams and notations** for modeling **object-oriented systems** and describes what the diagrams and symbols mean. It is also used to document business processes and workflows, and some types can **replace flowcharts** with a more standardized way of modeling and more features for readability.

Forward Design And Backward Design

	When The UML Is Drawn	Why
Forward design	**Before** coding the application	Gives a better view of the system or workflow, reveals design issues and flaws, and improves project health.
Backward design	**After** writing the code	The diagrams document the activities, roles, actors, and workflows.

Memory Aid
Forward = Before (F, B: "Forward Before"). **Backward = After** (B, A: "Back After").

Behavioral Versus Structural UML

	Behavioral Diagrams	Structural Diagrams
Depict	Elements that **depend on time** and convey the **dynamic** concepts	Elements that are **independent of time** and convey the concepts of the system
Natural-language hook	Elements resemble **verbs**; relationships convey the passage of time	Elements resemble **nouns**; relationships are structural or semantic
Vehicle reservation example	Make a Reservation, Rent a Car, Provide Credit Card Details	Car, Reservation, Driver's License, Credit Card, with connectors
Diagrams (the slides)	Activity, Use Case, Timing, State Machine, Sequence, Communication	Class, Object, Component, Deployment

Memory Aid
Behavioral = Verbs = time ("Be-have", action). Structural = Nouns = no time ("Structure", things).
Structural four: **C-O-C-D** = **C**lass, **O**bject, **C**omponent, **D**eployment ("Cats Obey Calm Dogs").
Behavioral six: **A**ctivity, **U**se case, **T**iming, **S**tate machine, **S**equence, **C**ommunication ("A Unicorn Tripped, Sliding Sideways, Crying").

The chapter cover slide shows the full classification: **structural** also includes Composite Structure, Package, and Profile diagrams (seven in all), and **behavioral** also includes the Interaction Overview diagram (seven in all). The content slides teach the six behavioral and four structural diagrams above.

Watch Out
Correct: "Composite Structure Diagram" and "Differentiate Behavioral UML diagrams **from** Structural". (The cover slide misspells "Composite Struture", and the outcome reads "differentiate ... to Structural".)

The Behavioral Diagrams

Diagram	What It Shows	Key Components And Points
Activity	The **flow of activities and actions**, sequential and in parallel, and how they connect. The focus is the set of activities, not what is produced. It has a **clear beginning and end**.	Objects used, consumed, or produced by an activity. Slide exercise: student **enrollment** process with an applicant and a registrar.
Use case	A user's **interaction with the system**; used to analyze **high-level requirements**.	**Functional requirements** = use cases (a **verb**, an action). **Actors** = human, organization, or internal or external application. **Relationships** = **straight arrows**.
Timing	How objects and actors act along a **linear time axis**, when time is the center of attention.	**Lifeline** (participant), **state timeline**, **duration constraint**, **time constraint**, **destruction occurrence** (ends a lifeline).
State machine (statechart)	The different **states of a component** and how it changes with **internal and external events**.	Called a state machine because it is a machine describing the states of an object.
Sequence	The **sequence of messages** between actors and objects in **chronological order**. Probably the most important UML diagram for design-level business models.	Actors or objects are active only when needed. **Vertical, top-down** message flow.
Communication	The **communication between objects**; the core components are the messages exchanged, like a sequence diagram.	Uses **number schemes and pointing arrows** for message flow instead of a vertical time order.

Memory Aid
Timing components, **L-S-D-T-D**: **L**ifeline, **S**tate timeline, **D**uration constraint, **T**ime constraint, **D**estruction occurrence.
Use case trio: **actors do use cases** (an actor acts, a use case is a verb, arrows connect them).
Sequence vs communication: **sequence is Vertical and by time, communication uses**

Numbers.

Watch Out
Sequence and communication diagrams both show messages. The difference is layout: sequence is **vertical, top-down chronological**; communication uses **numbered messages with arrows**.

The Structural Diagrams

Diagram	What It Shows	Key Points
Class	The **most common diagram for software documentation**, since most software is OOP based and OOP is based on classes and their relations.	Each class has **three fields**: the **class name** (top), **attributes** (below the name), and **operations/behaviors** (bottom). A connecting line shows the relation.
Object	Checks whether the **abstract structure** from the class diagram is viable when the classes are **instantiated**.	Focuses on a particular set of objects, attributes, and links. Slide example: the **Client** class gets a real client, **James**.
Component	How **components are wired together** to form larger components or software systems.	Breaks the system into smaller components to verify required functionality. Developers use it as a **roadmap for implementation** and for task assignment and skill planning.
Deployment	The **relation between software and hardware**: a physical model of how software components (**artifacts**) are deployed on hardware components (**nodes**).	Shows exactly **where each software component is deployed**.

Memory Aid
Class box = **N-A-O**: **N**ame on top, **A**ttributes in the middle, **O**perations at the bottom.
Deployment: **artifacts sit on nodes** (software on hardware).
Class is the **blueprint**, object is the **real instance** (class Client, object James).

Watch Out
Correct: "**Checking** Account", and common attributes such as balance and the debit and credit operations belong in the parent **Account** class. (The slide's class diagram says "Checkings Account" and repeats balance, debit(), and credit() in both subclasses.) The inheritance arrow has a **blank (hollow) head**.

The Slide Examples (Read From The Pictures)

The slides teach each diagram with one example. Recognize them by their story.

Diagram	Example On The Slide	What To Notice
Activity	Student **enrollment**: fill out enrollment forms, a decision on whether the forms are correct, enroll in the university, then two parallel branches (attend the overview presentation; enroll in seminars then pay the initial tuition).	Initial node (filled dot), decision diamond with guards like [correct], **fork bar** splits into parallel branches, **join bar** merges them, final node (bullseye).
Use case	**Inventory system**: actors Owner, Manager, Inspector, Supplier, Inventory Clerk; use cases such as Purchase Stock, Payment For Stock, Return Stock, Quality Control of Stock, Distribute/Sell Stock.	Actors sit outside the system and connect to use cases with straight lines. Dependencies are labeled **include** and **extend**.
Timing	**Human growth stages** on one lifeline (Neonate, Infant, Toddler, Play Age, Middle Childhood, Adolescence, Young and Middle and Late Adulthood, Death), each with a time constraint such as {0..28 days}.	Time runs left to right along a **linear time axis**; each stage is a state on the lifeline.
State machine	A **chess game**: Start, White's turn, Black's turn, with transitions "white moves" and "black moves", ending in Black wins, White wins, or Draw (checkmate, stalemate).	States are boxes, **events label the transitions**, and the game ends in final states.
Sequence	A **Registrar** deleting a student's course registration, sending messages to CourseRegPeriods, StudentCourseRegs, and Class, with return messages back.	Participants across the top, **time runs downward**, messages go in order.
Communication	An **online store**: Actor, Online Store, Categories, Item, Shopping Cart, Order, Inventory, with messages 1.* browse(), 1.1 filter(), 1.2 view_item(), 1.3 add_to_Cart(), 1.4 place_order(), 1.5 update_inventory().	Same messages as a sequence diagram but laid out as a **network of objects with numbered arrows**.
Class	A bank: **Client** (name, SSN, address, employment; owns()), super-class **Account**, and subclasses **Checking** and **Savings Account** joined by hollow-head inheritance arrows.	Three-part boxes. Subclasses inherit from the super-class.
Object	The same bank with real values: **James** (Client) and his two accounts, Student Checkings (balance 140.67) and Student Savings (balance 875.69, rate 0.15%).	Boxes show **instances with actual values**, not just attribute names.
Component	A **library system**: LMS database, Book, Search, and Member components wired by interfaces and dependencies, with a Transaction package.	Components connect through **provided and required interfaces** (ball and socket).

Deployment	A **hospital system**: Desktop client, Hospital local server, Printer, and Data base server as nodes, linked by network paths labeled primary network or private network.	3D boxes are **nodes**, lines are communication paths.

Watch Out
Correct: the Savings object in the object diagram is an instance of **Savings Account**. (The slide's box reads "Student Savings: Checkings Account".) Also, an extend arrow points from the **extending** use case to the **base** use case, and a Death state has no fixed duration, which is why its constraint reads "Unpredictable" (spelled "Unpredicatble" on the slide).

Part 3: Module 7

Module 7, Subtopic 1: Strategic Trends In Software Engineering

Three objectives: (1) understand **AI-native and LLM-based applications**, (2) differentiate a **GenAI platform** from **open GenAI ecosystems**, (3) discuss **Green Software Engineering** and the trend of **maximizing talent density**.

AI-Native Software Engineering

AI-Native Software Engineering means building software systems where **AI is a core part of the design, development, and operation, not just an added feature**. It integrates AI models directly into the **SDLC**, helping systems learn, adapt, and improve automatically.

Four Key Ideas

Key Idea	Meaning	Example On The Slide
AI-driven design and architecture	Uses machine learning to help design better system structures automatically.	AI suggests the most efficient database schema or API structure from project data.
Continuous learning and improvement	Software evolves by learning from user feedback and new data.	A recommendation system that gets smarter the more users interact with it.
Automation of repetitive tasks	AI automates coding, testing, debugging, and deployment.	GitHub Copilot and Jenkins AI plugins help write and test code faster.
Ethical and responsible AI	Makes sure the AI in the software is fair, transparent, and unbiased.	(No example on the slide.)

Tools And Frameworks

Category	Tool	Job
Machine learning frameworks	TensorFlow, PyTorch	Build and train machine learning models efficiently.
Lifecycle management	MLflow	Manage the machine learning model lifecycle, including experimentation and deployment.
Pipeline orchestration	Kubeflow	Create and manage machine learning pipelines on Kubernetes for scalable, large-scale AI workflows.
AI-assisted coding	GitHub Copilot	Improves developer productivity by suggesting code snippets and automating tasks.

Memory Aid
Key ideas = **D**-**L**-**A**-**E**: **D**esign, **L**earning, **A**utomation, **E**thics ("Dogs Like Apple Eating").

Tools = **B-M-O-A**: **B**uild/train (TensorFlow, PyTorch), **M**anage lifecycle (MLflow), **O**rchestrate (Kubeflow), **A**ssist coding (Copilot).

Watch Out
Correct: Kubeflow runs on **Kubernetes**. (The slide spells it "Kubemeters".)

LLM-Based Applications And Agents

LLMs (Large Language Models) are **AI systems trained on large datasets to understand and generate human-like language** effectively. If a question asks for this definition in words, the answer is **Large Language Models** (do not abbreviate).

Practical Uses Of LLMs In Software

Use	What It Does	Example
1. Chatbots and virtual assistants	Handle customer support or developer queries automatically.	An AI chatbot that helps users troubleshoot app issues.
2. Text summarization and translation	Summarize technical documents or translate documentation for global teams.	
3. Code generation and debugging	Write, explain, or fix code in many languages.	ChatGPT suggesting optimized Python code or detecting bugs.
4. Content creation and search	Create documentation, release notes, or test cases. Improve search and knowledge retrieval.	
5. Research and workflow automation	LLM agents gather research data, summarize results, and automate routine development tasks.	

Applications Of LLM-Based Agents

Application	What The Agent Does
1. Customer support	Automates responses to common customer queries, improving response times and reducing workload.
2. Intelligent tutoring systems	Gives personalized learning by adapting content to each student's needs.
3. Research assistance	Summarizes articles, extracts insights, and generates hypotheses.
4. Workflow automation	Automates repetitive tasks and coordinates multiple systems.
5. Personalized content generation	Creates tailored articles, emails, and marketing materials from user preferences and data.

Memory Aid
LLM uses: **Chat, Summarize, Code, Create, Research** ("Cats Sit Calmly, Creating Rugs").
Agent applications: **S-T-R-W-P**: **S**upport, **T**utoring, **R**esearch, **W**orkflow, **P**ersonalized content ("Some Tigers Read Wild Poems").

Watch Out
An LLM agent summarizes articles and extracts insights. It does **not** repair hardware, manufacture processors, or install network cables.

GenAI Platform Engineering

Generative AI systems create diverse content like **text, images, and code** using advanced AI models.

Core Components

Component	Role
1. Role of Large Language Models	The foundation for content understanding and generation: summarization, translation, and conversation.
2. APIs for integration	Let the platform **interact with external tools and services**, extending its capabilities and integration with other systems.
3. Data pipelines management	Handle **ingestion, processing, and storage** of data so models get relevant, high-quality information.

Applications Across Industries

Application	What It Does
Customer support chatbots	Instant, accurate responses; better customer experience and lower costs.
Marketing content generation	Rapid creation of articles, ads, and social media content.
Software development assistance	Code assistants help write, debug, and optimize code.
Data analysis and summarization	Help analysts extract insights and generate reports.

Memory Aid
Platform = **L-A-D**: **L**LMs (the brain), **A**PIs (the hands, they reach outside tools), **D**ata pipelines (the food).

Watch Out
APIs do **not** prevent outside interaction, eliminate data pipelines, or replace AI models. They integrate the platform with external tools and services.

Maximizing Talent Density

Talent density measures the **proportion of high-performing individuals** within a workforce. It focuses on **quality over quantity**.

Strategies To Maximize It	Measuring It
1. Strategic hiring: recruit top performers who match company values and goals.	**1. Performance reviews and feedback:** show strengths and areas to improve.
2. Employee development: training and mentorship for continuous growth.	**2. Skills assessment and benchmarking:** identify top performers and skill gaps against standards.
3. Culture of excellence: accountability and high performance.	**3. Engagement and satisfaction surveys:** measure engagement and alignment.
4. Managing underperformance: constructive feedback and tough decisions.	**4. Retention and promotion metrics:** track retention and promotion of high performers.

Memory Aid
Strategies = **H-D-C-M**: **H**ire, **D**evelop, **C**ulture, **M**anage underperformers.
Measures = **P-S-E-R**: **P**erformance reviews, **S**kills assessment, **E**ngagement surveys, **R**etention and promotion.

Open GenAI Ecosystems

Open GenAI promotes **transparent collaboration by sharing models, datasets, and tools** among developers worldwide. An open environment that promotes collaboration through sharing AI models, datasets, and tools is exactly this definition.

Key Component	What It Is
1. GenAI models	Large language, vision, and multimodal models that process diverse data types.
2. Development tools	Training frameworks, evaluation suites, and preprocessing utilities.
3. Hosting platforms	Deployment, collaboration, and workflow management for GenAI projects at scale.

Topic	Slide Content
Benefits	Foster **collaboration, transparency, and inclusivity**, accelerating AI innovation and access.
Key success factors	**Interoperability, community engagement, and responsible governance.**
Future challenges and investments	Address **quality, security, and sustainability** with ongoing investment in infrastructure and ethics.

Memory Aid
Components: **M-D-H** = **M**odels, **D**evelopment tools, **H**osting.
Success factors: **I-C-G** = **I**nteroperability, **C**ommunity, **G**overnance.

Challenges: **Q-S-S** = **Q**uality, **S**ecurity, **S**ustainability.

Green Software Engineering

Green Software Engineering focuses on **designing software to minimize environmental impact** and promote **sustainability**. It does **not** aim to increase environmental impact.

Energy-Efficient Coding Practice	How It Saves Energy
1. Efficient algorithms and data structures	Lower computational complexity and memory use, so lower energy use.
2. Code optimization techniques	Optimizing loops and avoiding unnecessary computation decreases power use.
3. Profiling to identify bottlenecks	Profiling and benchmarking find performance issues for targeted improvements.
4. Scalable and sustainable code	Efficient resource use under varying loads.

Application Of Green Software	What It Does
1. Carbon-aware cloud services	Adjust workloads based on **carbon intensity** to optimize resources and cut emissions.
2. Energy-efficient AI training	Green AI improves training efficiency using optimized hardware and algorithms.
3. Energy-saving mobile apps	Use less battery and extend device lifespan.
4. Sustainable software development	Improves operational efficiency and stakeholder engagement.

Memory Aid
Practices = **A-O-P-S**: **A**lgorithms, **O**ptimize, **P**rofile, **S**calable ("Always Open Pretty Skies").
Applications = **C-E-E-S**: **C**arbon-aware cloud, **E**nergy-efficient AI, **E**nergy-saving mobile, **S**ustainable development.

One-Page Cram Sheet

Topic	Remember
Object	Properties, behaviors, states
OOA	Models functional requirements with objects, independent of implementation. Five activities: Find, Organize, Describe interactions, External behavior, Internal behavior
OOD	Extends OOA and adds implementation constraints (platform, time, budget, performance, skills)
OOAD	Method based on a system's object models. Good: encapsulation, easy to understand. Bad: ill-suited to procedural apps, too complex for simple apps
Procedural	Step-by-step instructions, top-down, imperative. Data exposed to the whole program. Parameter passing: value, reference, result, value-result, name
Procedural vs OOP	Functions vs objects, top-down vs bottom-up, no access specifiers vs private/public/protected, no data hiding vs data hiding
Methodology	Steps to form, plan, and control system development. Nine criteria. Tools (language, IDE, CASE) and techniques (best practices)
Six models	Waterfall, Iterative, DevOps, V-Model, Spiral, Prototyping
Waterfall / V / Spiral	Rigid requirements first / waterfall bent up with testing pairs / risk-driven in four quadrants
Iterative / DevOps / Prototype	Small fixed-scope increments, less customer involvement / Dev + Ops continuous delivery / early model with feedback
Strategy	No one-size-fits-all. A methodology does not guarantee success. Build in flexibility
UML	Standard way to visualize system design. Forward = before coding, backward = after coding
Behavioral (verbs, time)	Activity, Use case, Timing, State machine, Sequence, Communication
Structural (nouns, no time)	Class, Object, Component, Deployment
AI-native	AI is core to design, development, and operation. Four key ideas: design, learning, automation, ethics
AI tools	TensorFlow and PyTorch (build), MLflow (lifecycle), Kubeflow (pipelines), GitHub Copilot (coding)
LLM	Large Language Models: trained on large datasets to understand and generate human-like language
GenAI platform	LLMs, APIs (integrate with external tools and services), data pipelines
Talent density	Proportion of high performers; quality over quantity
Open GenAI	Shares models, datasets, and tools. Components: models, tools, hosting. Success: interoperability, community, governance
Green SE	Minimize environmental impact: efficient algorithms, optimization, profiling, scalable code